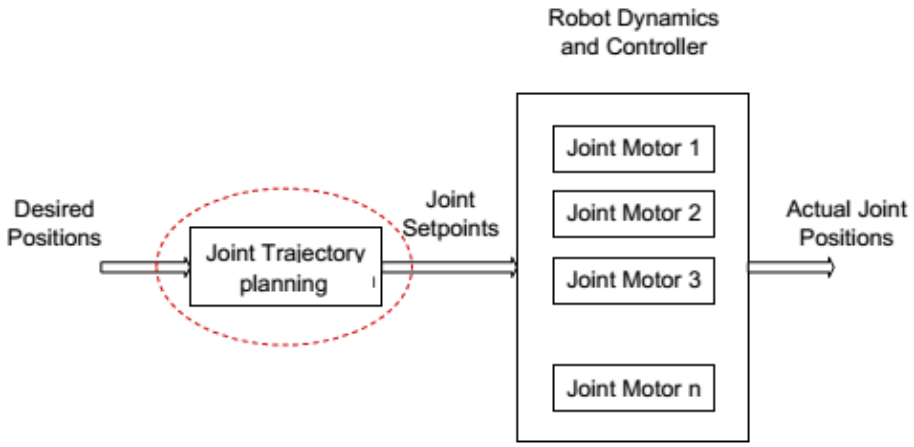
PRAKTIKUM 5
TRAJECTORY PLANNING (I)

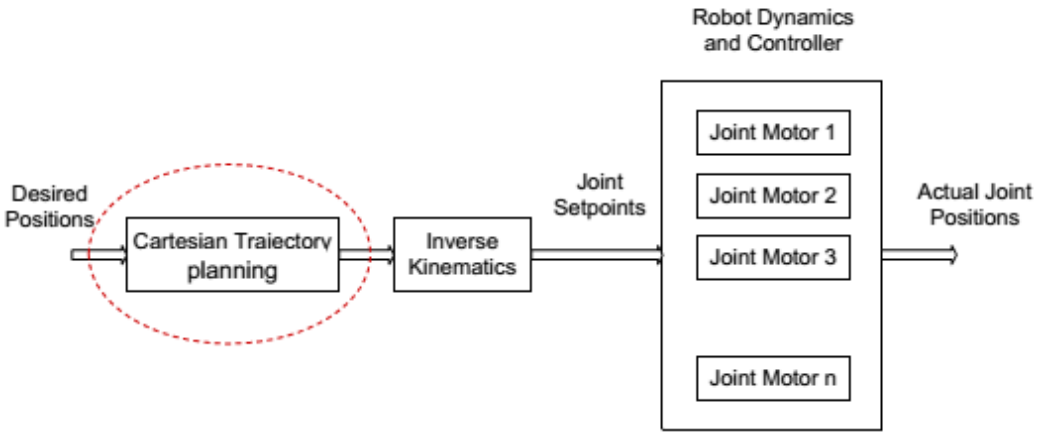
Pendahuluan

Pada robot manipulator, untuk menyelesaikan permasalahan yang terkait dengan *trajectory generation* terdapat dua pendekatan yang dapat dilakukan yaitu berdasarkan *joint position* dan *work position*. Pendekatan *joint position* akan menghasilkan besaran dari setiap joint dari manipulator tersebut. Pada pendekatan *work position* akan menghasilkan ouput berupa posisi dari *end of effector* dari manipulator yang digunakan. Pada pendekatan secara *work position* diperlukan adanya mekanisme *inverse kinematic* untuk mengubah output dari proses *trajectory planning* dalam posisi menjadi besaran nilai untuk setiap *joint*-nya.

1. *Joint Position Based Approach*



2. *Work Position Based Approach*



First Order Polynomial

Untuk menyelasaikan permasalahan sederhana dari proses *trajectory generation* diantara dua titik posisi yaitu posisi awal (q_0) dan posisi akhir (q_f) maka dapat diselesaikan menggunakan prinsip dari *first order* polynomial sehingga akan didapatkan posisi yang berada diantara q_0 dan q_f dalam rentang tertentu.

Persamaan dari posisi adalah sebagai berikut :

$$q(t) = a_1 t + a_0 \quad (1)$$

Subtitusikan persamaan (1) terhadap waktu awal dan waktu final

$$t = 0 \rightarrow q(0) = a_1 0 + a_0 \quad (2)$$

$$t = t_f \rightarrow q(t_f) = a_1 t_f + a_0 \quad (2)$$

Posisi awal dan final

$$t = 0 \rightarrow q(0) = q_s \quad (3)$$

$$t = t_f \rightarrow q(t_f) = q_f \quad (3)$$

Dari persamaan (2) dan (3) maka

$$a_0 = q_s \quad (4)$$

$$q_f = a_1 t_f + q_s \rightarrow \ a_1 = \frac{q_f - q_s}{t_f} \quad (4)$$

Sehingga

$$q(t) = \frac{q_f - q_s}{t_f} t + q_s \quad (5)$$

$Tujuan$

1. Mahasiswa dapat memahami *trajectory planning* pada robot lengan
2. Mahasiswa dapat menghasilkan *trajectrory* menggunakan persamaan *first order polynomial*

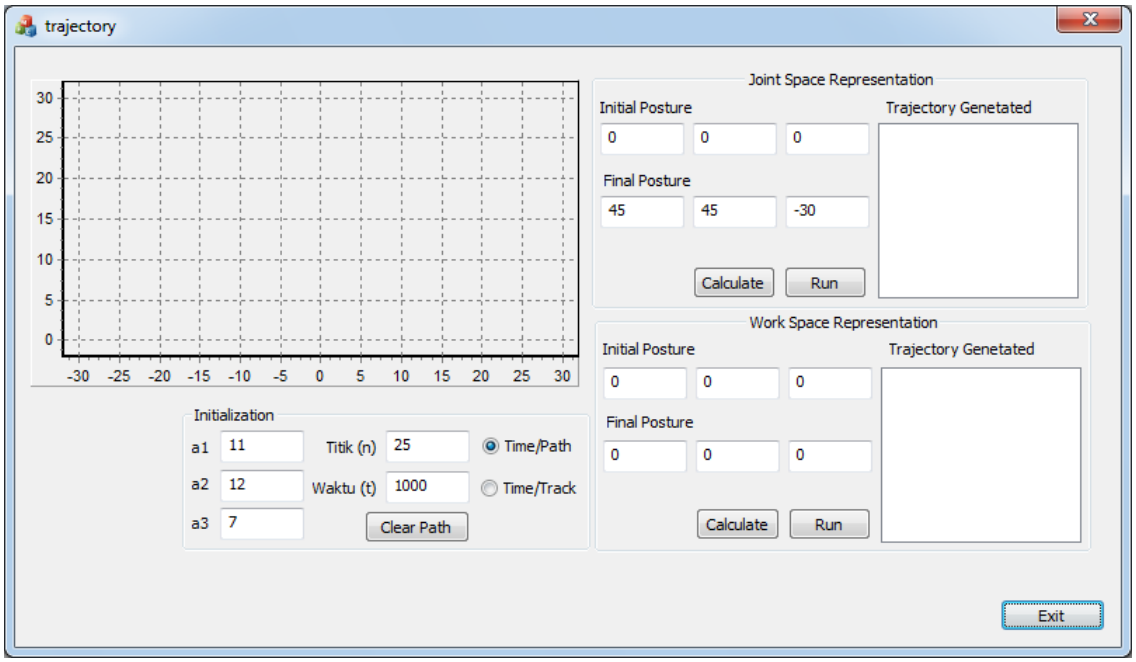
$Alat\ dan\ Bahan$

1. Modul AL5A Arm Robot
2. Modul SSC-32 Lynxmotion
3. Ms. Visual C++
4. Penggaris dan Busur
5. ActiveX Control Teechart

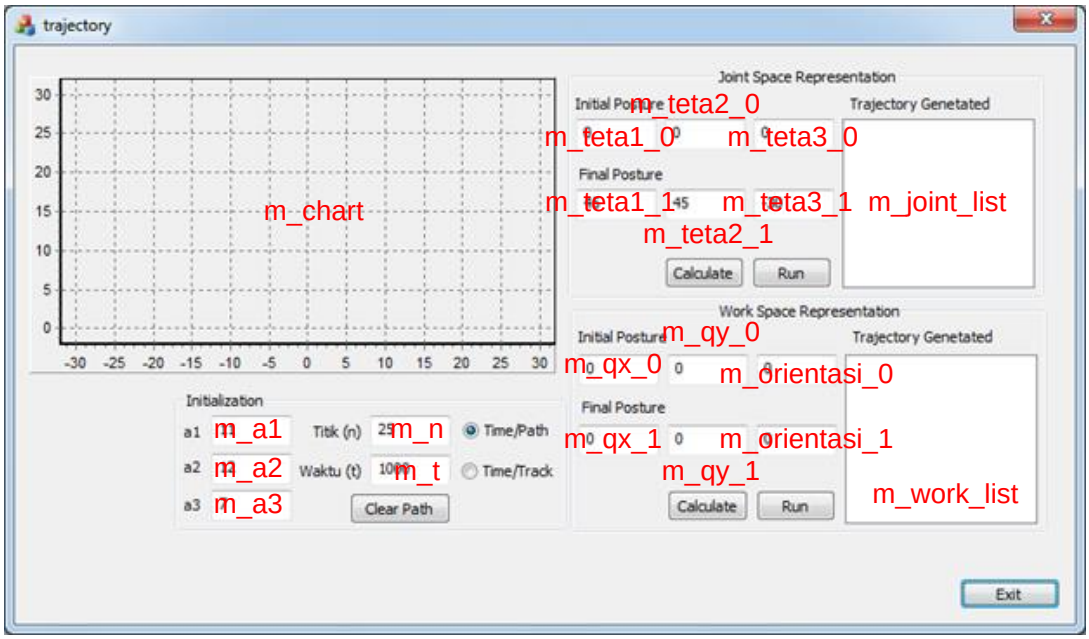
6. ActiveX Control MSCOMM32.ocx

Prosedur Percobaan

1. Buatlah dialog seperti gambar berikut :



2. Atur *member variable* seperti gambar berikut :



3. Buat fungsi untuk menggambarkan lengan pada *chart* sebagai berikut :

```
void CtrajectoryDlg::ArmDraw(float sdt1,float sdt2,float sdt3){
	int a,a1,a2,a3;
	float kx,ky,px,py,qx,qy;
	sdt1=sdt1 * (3.14/180);
	sdt2=sdt2 * (3.14/180);
	sdt3=sdt3 * (3.14/180);
	UpdateData(1);
	kx=m_a1*cos(sdt1);
	ky=m_a1*sin(sdt1);
	px=m_a2*cos(sdt1+sdt2) + kx;
	py=m_a2*sin(sdt1+sdt2) + ky;
	qx=m_a3*cos(sdt1+sdt2+sdt3) + px;
	qy=m_a3*sin(sdt1+sdt2+sdt3) + py;
	UpdateData(0);
	((CSeries)m_chart.Series(0)).Clear();
	((CSeries)m_chart.Series(1)).Clear();
	((CSeries)m_chart.Series(2)).Clear();
	((CSeries)m_chart.Series(0)).AddXY(0,0,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(0)).AddXY(kx,ky,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(1)).AddXY(kx,ky,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(1)).AddXY(px,py,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(2)).AddXY(px,py,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(2)).AddXY(qx,qy,NULL,RGB(100,100,100));
	((CSeries)m_chart.Series(3)).AddXY(qx,qy,NULL,RGB(100,100,100));
}
```

4. Buat fungsi *Inverse Kinematic* mengacu pada praktikum sebelumnya. Berikut list programnya :

```
void CtrajectoryDlg::InverseKinematic1(float iqx,float iqy,float iorientasi){
	float teta1,teta2,teta3,ipx,ipy;
	UpdateData(1);
	ipx=iqx - m_a3*cos(iorientasi*3.14/180);
	ipy=iqy - m_a3*sin(iorientasi*3.14/180);
	teta2=(pow(ipx,2)+pow(ipy,2)-pow((float)m_a1,2)-pow((float)m_a2,2)) /
(2*(float)m_a1*(float)m_a2);
	if(teta2>=1.0)		teta2=1.0;
	else if(teta2<=-1.0) teta2=-1.0;
	teta2=acos(teta2);
	teta1=((float)m_a2*sin(teta2)) / ((float)m_a1+(float)m_a2*cos(teta2));
	teta1=atan2(ipy,ipx) - atan(teta1);
	teta2=(teta2*(180/3.14));
	teta1=teta1*(180/3.14);
	teta3=iorientasi - teta2 - teta1;
	if(teta1<0)		teta1=0;
	else if(teta1>180)	teta1=180;
	if(teta2<0)		teta2=0;
	else if(teta2>180)	teta2=180;
	if(teta3<0)		teta3=0;
	else if(teta3>180)	teta3=180;
	ArmDraw(teta1,teta2,teta3);
	UpdateData(0);
}
```

5. Pada *group Joint Space Representation*, pada button *Calculate* tambahkan program :

```
void CtrajectoryDlg::OnBnClickedButton1() // Joint Space
{
	CString aa;
	int i;

	m_joint_list.ResetContent();
	UpdateData(1);
	for(i=0;i<=m_n;i++){
		teta1[i]=m_teta1_0 + ((m_teta1_1-m_teta1_0)/(float)(m_n)*(float)i);
		teta2[i]=m_teta2_0 + ((m_teta2_1-m_teta2_0)/(float)(m_n)*(float)i);
		teta3[i]=m_teta3_0 + ((m_teta3_1-m_teta3_0)/(float)(m_n)*(float)i);
		aa.Format(_T("%3d  %3.1f  %3.1f  %3.1f"),i,teta1[i],teta2[i],teta3[i]);
		m_joint_list.AddString(aa);
	}
	UpdateData(0);
}
```

6. Pada *group Joint Space Representation*, Pada button *Run* tambahkan program :

```
void CtrajectoryDlg::OnBnClickedButton3() // Joint Space
{
	int time;
	n=0; space=0;
	UpdateData(1);
	if(jenis==0) time=m_t;
	else		time=m_t/m_n;
	UpdateData(0);
	SetTimer(0,time,NULL);
}
```

7. Pada *group Work Space Representation*, Pada *button Calculate* tambahkan program :

```
void CtrajectoryDlg::OnBnClickedButton2() // Work Space
{
	CString aa;
	int i;

	m_work_list.ResetContent();
	UpdateData(1);
	for(i=0;i<=m_n;i++){
		qx[i]=m_qx_0 + ((m_qx_1-m_qx_0)/(float)(m_n)*(float)i);
		qy[i]=m_qy_0 + ((m_qy_1-m_qy_0)/(float)(m_n)*(float)i);
		orientasi[i]=m_orientasi_0 + ((m_orientasi_1-
m_orientasi_0)/(float)(m_n)*(float)i);
		aa.Format(_T("%3d  %3.1f  %3.1f  %3.1f"),i,qx[i],qy[i],orientasi[i]);
		m_work_list.AddString(aa);
	}
	UpdateData(0);
}
```

8. Pada *group Work Space Representation*, Pada *button Run* tambahkan program :

```
void CtrajectoryDlg::OnBnClickedButton4() // Work Space
{
	int time;
	n=0; space=1;
	UpdateData(1);
	if(jenis==0) time=m_t;
	else		time=m_t/m_n;
	UpdateData(0);
	SetTimer(0,time,NULL);
}
```

9. Pada *buton Clear Path* tambahkan program :

```
void CtrajectoryDlg::OnBnClickedButton5()
{
	((CSeries)m_chart.Series(3)).Clear();
}
```

10. Pada *radio button Time/Path* tambahkan program berikut :

```
void CtrajectoryDlg::OnBnClickedRadio1()
{
	jenis=0;
}
```

11. Pada *radio button Time/Track* tambahkan program berikut :

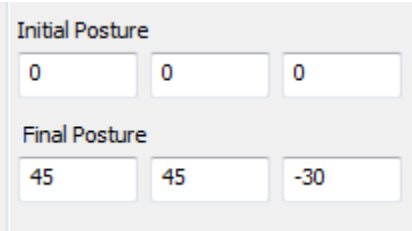
```
void CtrajectoryDlg::OnBnClickedRadio2()
{
	jenis=1;
}
```

12. Tambahkan *Event Handler* WM_TIMER dengan cara masuk ke *dialog Class Wizard* pilih *tab Messages* dan cari WM_HANDLER.
13. Tambahkan program pada *Event Handler Timer* sebagai berikut :

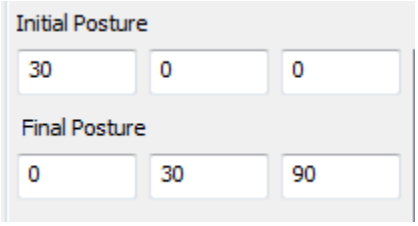
```
void CtrajectoryDlg::OnTimer(UINT_PTR nIDEvent)
{
	if(space==0)		ArmDraw(teta1[n],teta2[n],teta3[n]);
	else if(space==1)	InverseKinematic1(qx[n],qy[n],orientasi[n]);
	n++;
	UpdateData(1);
	if(n>m_n) KillTimer(0);
	UpdateData(0);

	CDialogEx::OnTimer(nIDEvent);
}
```

14. Jalankan program yang telah anda buat, pada *group Joint space* berikan nilai pada *edit box* dari *Initial Posture* dan *Final Posture* seperti gambar berikut :



15. Isi nilai pada *edit box* Titik (n) sebesar 5 dan waktu sebesar 1000 dan *radio button* pada posisi *Time/Path*.
16. Jalankan dengan mengklik *Calculate* kemudian *Run* dan amati hasilnya !
17. Ganti *radio button* yang aktif pada *Time/Track*
18. Jalankan dengan mengklik *Calculate* kemudian *Run* dan amati hasilnya !
19. Jalankan program yang telah anda buat, pada *group work space* berikan nilai pada *edit box* dari *Initial Posture* dan *Final Posture* seperti gambar berikut :



20. Isi nilai pada *edit box* Titik (n) sebesar 5 dan waktu sebesar 1000 dan *radio button* pada posisi *Time/Path*.
21. Jalankan dengan mengklik *Calculate* kemudian *Run* dan amati hasilnya !
22. Ganti radio *button* yang aktif pada *Time/Track*
23. Jalankan dengan mengklik *Calculate* kemudian *Run* dan amati hasilnya !

Setelah program diatas selesai, lakukan pengujia terlebih dahulu apakah program sudah berjalan sebagaimana mestinya dengan memperhatikan simulasi pergerakan pada cart. Setelah dipastikan berjalan sebagaimana mestinya maka program yang telah dibuat akan dihubungkan dengan robot lengan AL5A. tambahkan perintah berikut dalam program *trajectory planning* :

1. Tambahkan program pada *Event Handler Timer* sebagai berikut :

```
void CtrajectoryDlg::OnTimer(UINT_PTR nIDEvent)
{
	if(space==0)		ArmDraw(teta1[n],teta2[n],teta3[n]);
	else if(space==1)	InverseKinematic1(qx[n],qy[n],orientasi[n]);
	n++;
	UpdateData(1);
	if(n>m_n) KillTimer(0);

	UpdateData(1);

	m_slider.SetPos((int)m_satu);
	m_slider2.SetPos((int)m_dua+90);
	m_slider3.SetPos((int)m_tiga+90);

	UpdateData(0);

	CDialogEx::OnTimer(nIDEvent);
}
```

2. Jalankan dengan mengklik *Calculate* kemudian *Run* dan amati hasilnya.

Data dan Analisis

1. Untuk *Joint space representation* sajikan data besar sudut tiap *joint* dalam grafik 2 dimensi (*plot*) untuk setiap *trajectory* yang dibuat.
2. Untuk *work space representation* sajikan data posisi qx, qy dan Orientasi dalam grafik sumbu kartesian 2 dimensi untuk setiap *trajectory* yang dibuat.
3. Dapatkan *error* dari qx dan qy (dihitung menggunakan *Euclidean distance*) untuk masing-masing titik (*path*) yang dihasilkan tiap satuan waktu (t).
4. Berdasarkan error yang terjadi dari masing-masing percobaan lakukan analisis dengan mencari katerkaitan dan tren *error* yang terjadi.

Kesimpulan

Berdasarkan hasil analisis, buatlah kesimpulan yang berkaitan dengan percobaan yang telah anda lakukan.

Tugas

1. Lakukan percobaan dengan memberikan parameter *Initial Posture* dan *Final Posture* yang berbeda sebanyak 5 kali, catatlah *trajectory* yang dihasilkan!
2. Jelaskan perbedaan antara *Joint Space* dan *Work Space* berdasarkan percobaan yang telah anda lakukan!
3. Jelaskan pengaruh dari pemberian nilan pada Titik(n) dan Waktu(t) yang berbeda-beda!
4. Jelaskan perbedaan antara pemilihan *Radio Button Time/Path* dan *Time/Track*!